

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 Математика и компьютерные науки

ОТЧЕТ ПО Лабораторной работе №3
по дисциплине Программирование микроконтроллеров

События и прерывания.

Студент,
группы 5130201/40003

_____ Адиатуллин Т.Р

Руководитель,
Преподаватель

_____ Вербова Н. М.

«_____» _____ 2026 г.

Санкт-Петербург, 2026

Содержание

1	Цель работы	3
2	Постановка задачи	3
3	Алгоритм работы программы	3
3.1	Инициализация проекта	3
3.2	Настройка портов GPIOG и GPIOA	4
3.3	Настройка внешних прерываний (EXTI)	4
3.4	Настройка и активация прерываний в NVIC	4
3.5	Реализация мигания светодиодов	4
3.6	Используемые переменные и регистры	5
	Заключение	9

1. Цель работы

Ознакомится с основными методами обработки событий и прерываний.

2. Постановка задачи

разработать программу для микроконтроллера (МК) STM32F200 мигающую светодиодом PG7 и регистрирующую и обрабатывающую с разным приоритетом замыкание кнопок “WAKEUP” и “USER”. При нажатии кнопки “WAKEUP” на некоторое время должен подключаться светодиод PG6, а при нажатии кнопки “USER” светодиод PG8 (см. рис. 1).

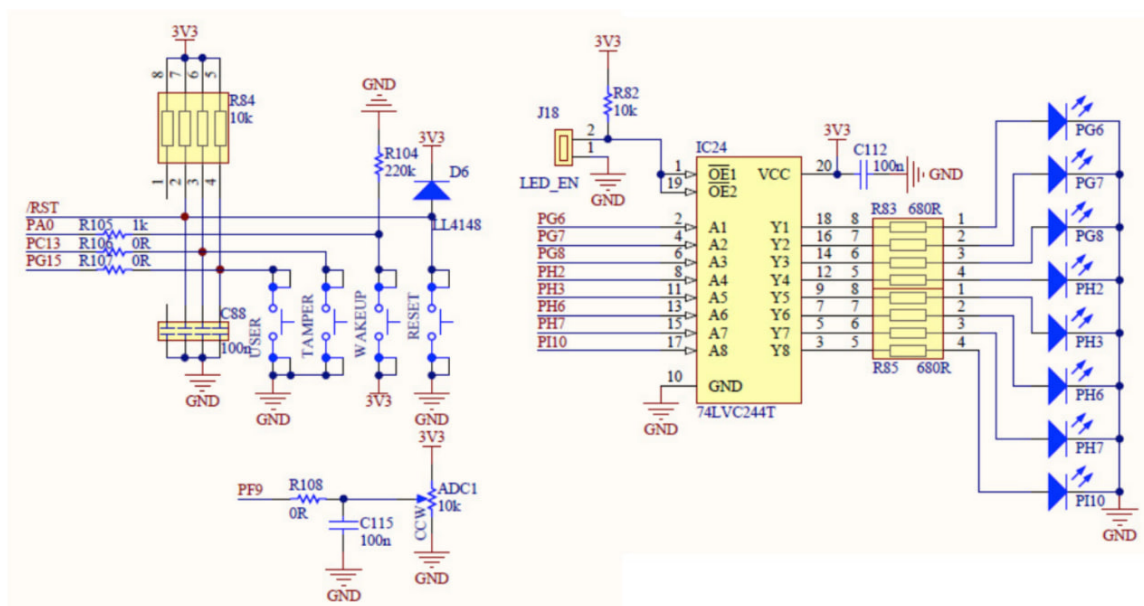


Рис. 1: Схема блоков кнопок и светодиодов

3. Алгоритм работы программы

3.1. Инициализация проекта

- Создание нового проекта в среде Keil μ Vision5.
- Выбор микроконтроллера **STM32F207IGHx**.
- Настройка параметров проекта, включая поддержку ядра (*CMSIS CORE*), запуск (*Startup*) и настройки HAL.
- Подключение библиотеки `stm32f2xx.h`, где хранится информация обо всех адресах регистров.

3.2. Настройка портов GPIOG и GPIOA

- Включение тактирования порта GPIOG через регистр RCC->AHB1ENR.
- Настройка пинов PG6, PG7, PG8 как выходы через GPIOG->MODER.
- Установка типа выхода в тяни-толкай (push-pull) через GPIOG->OTYPER.
- Установка низкой скорости переключения для порта GPIOG через GPIOG->OSPEEDR.
- Настройка всех пинов GPIOA как входы через GPIOA->MODER.
- Аналогичная настройка типа выхода и скорости переключения для GPIOA.

3.3. Настройка внешних прерываний (EXTI)

- Включение тактирования контроллера конфигурации системы (SYSCFG) через RCC->APB2ENR.
- Разрешение прерываний для линий EXTI0 (PA0) и EXTI15 (PG15) через EXTI->IMR.
- Настройка прерывания EXTI0 по нарастающему фронту (0 → 1) через EXTI->RTSR.
- Настройка прерывания EXTI15 по спадающему фронту (1 → 0) через EXTI->FTSR.
- Связывание EXTI0 с PA0, а EXTI15 с PG15 через SYSCFG->EXTICR.

3.4. Настройка и активация прерываний в NVIC

- Установка групповых приоритетов прерываний через NVIC_SetPriorityGrouping.
- Установка приоритетов для EXTI0 и EXTI15_10.
- Активация обработчиков прерываний EXTI0 и EXTI15_10 через NVIC_EnableIRQ.

3.5. Реализация мигания светодиодов

1. Основной цикл программы:

- Постоянное мигание PG7 с задержкой.

2. Обработчик прерывания EXTI0 (нажатие на PA0):

- 10-кратное мигание PG6.
- Очистка флага прерывания EXTI0.

3. Обработчик прерывания EXTI15 (нажатие на PG15):

- 10-кратное мигание PG8.
- Очистка флага прерывания EXTI15.

3.6. Используемые переменные и регистры

RCC->AHB1ENR — включает тактирование GPIOG.

RCC->APB2ENR — включает тактирование контроллера конфигурации (SYSCFG).

GPIOG->MODER — задает режим работы пинов PG6, PG7, PG8 (выход).

GPIOG->OTYPER — определяет тип выхода (push-pull).

GPIOG->OSPEEDR — определяет скорость переключения (низкая).

GPIOA->MODER — все пины настроены как входы.

EXTI->IMR — маска разрешенных внешних прерываний (EXTI0, EXTI15).

EXTI->RTSR — определяет срабатывание EXTI0 по нарастающему фронту.

EXTI->FTSR — определяет срабатывание EXTI15 по спадающему фронту.

SYSCFG->EXTICR — связывает EXTI0 с PA0, а EXTI15 с PG15.

NVIC_SetPriorityGrouping — устанавливает групповой приоритет прерываний.

NVIC_SetPriority — задает приоритет конкретных прерываний.

NVIC_EnableIRQ — включает прерывания EXTI0 и EXTI15_10.

GPIOG->ODR — управляет состоянием светодиодов.

EXTI->PR — очищает флаг прерывания после его обработки.

Код программы

```
1 #include <stm32f207xx.h>
2 #include <stm32f2xx.h>
3 #include <core_cm3.h>
4
5 // Функция задержки
6 // Просто крутит пустой цикл, чтобы процессор был чемто- занят
7 // Чем больше число t, тем дольше ждать
8 void delay (unsigned long t)
9 {
10     unsigned long i;
11     for (i = 0; i < t; ++i);
12 }
13
14 int main (void)
15 {
16     // Включаем тактирование
17     // Чтобы порты G и A заработали, на них нужно подать тактовый
    сигнал
18     // Бит 6 включает порт G, Бит 0 включает порт A
19     RCC->AHB1ENR |= (RCC_AHB1ENR_GPIOGEN | RCC_AHB1ENR_GPIOAEN);
20
21     // Настройка светодиодов на порту G
22     // Настраиваем пины 6, 7 и 8 порта G как выходы
23     // Это нужно, чтобы мы могли подавать на них напряжение и
    зажигать диоды
24     GPIOG->MODER |= (GPIO_MODER_MODE6_0 | GPIO_MODER_MODE7_0 |
    GPIO_MODER_MODE8_0);
25
26     // Настраиваем тип выхода как обычный
27     GPIOG->OTYPER = 0;
28
29     // Скорость переключения пинов ставим низкую
30     // Для мигания светодиодом высокая скорость не нужна
31     GPIOG->OSPEEDR = 0;
32
33     // Настройка кнопки на порту A
34     // Настраиваем весь порт A на вход
35     // Микроконтроллер будет слушать напряжение на ножке
36     GPIOA->MODER = 0;
37     GPIOA->OTYPER = 0;
38     GPIOA->OSPEEDR = 0;
39
40     // Включаем тактирование системного контроллера SYSCFG
41     // Он нужен, чтобы подключить ножку порта к линии прерывания
42     RCC->APB2ENR |= RCC_APB2ENR_SYSCFGEN;
43
44     // Связываем ножки с линиями прерываний
45     // Линия 0 будет слушать порт A, это пин PA0
46     SYSCFG->EXTICR[0] |= SYSCFG_EXTICR1_EXTIO_PA;
47
```

```

48 // Линия 15 будет слушать порт G, это пин PG15
49 SYSCFG->EXTICR[3] |= SYSCFG_EXTICR4_EXTI15_PG;
50
51 // Настройка самих прерываний EXTI
52 // Разрешаем прерывания на линиях 0 и 15
53 // Если этого не сделать, процессор будет игнорировать сигналы
54 EXTI->IMR |= (EXTI_IMR_MR0 | EXTI_IMR_MR15);
55
56 // Настраиваем срабатывание по переднему фронту для линии 0
57 // Прерывание сработает, когда сигнал пойдет вверх
58 EXTI->RTSR |= EXTI_RTSR_TR0;
59
60 // Настраиваем срабатывание по заднему фронту для линии 15
61 // Прерывание сработает, когда сигнал пойдет вниз
62 EXTI->FTSR |= EXTI_FTSR_TR15;
63
64 // Настройка приоритетов
65 // Задаем группировку приоритетов, это схема разделения
важности задач
66 NVIC_SetPriorityGrouping (0x5UL);
67
68 // Устанавливаем важность для прерывания кнопки PA0
69 // Чем меньше число, тем важнее прерывание
70 NVIC_SetPriority (EXTI0_IRQn, 2 | 2 << 2);
71
72 // Устанавливаем важность для прерывания кнопки PG15
73 // Это прерывание будет важнее предыдущего
74 NVIC_SetPriority (EXTI15_10_IRQn, 1 | 1 << 2);
75
76 // Активируем эти прерывания в самом процессоре
77 // Теперь процессор готов реагировать на события
78 NVIC_EnableIRQ (EXTI0_IRQn);
79 NVIC_EnableIRQ (EXTI15_10_IRQn);
80
81 // Главный цикл программы
82 // Пока никто не нажимает кнопки, процессор просто мигает
диодом на пине 7
83 for (;;) {
84     // Меняем состояние пина 7 на противоположное
85     GPIOG->ODR ^= GPIO_ODR_OD7;
86     delay (200000);
87 }
88
89
90 // Обработчик прерывания для линии 0, когда нажали PA0
91 // Эта функция вызывается сама аппаратно
92 void EXTI0_IRQHandler (void)
93 {
94     unsigned int i;
95     // Мигаем другим диодом PG6 быстро 60 раз
96     for (i = 0; i < 60; ++i) {
97         GPIOG->ODR ^= GPIO_ODR_OD6;
98         delay (100000);

```

```

99     }
100
101     // Сбрасываем флаг прерывания
102     // Мы пишем 1 в бит PR0, чтобы подтвердить выполнение
103     EXTI->PR |= EXTI_PR_PR0;
104 }
105
106 // Обработчик прерывания для линий 10-15, когда сработал PG15
107 // Эта функция вызывается сама аппаратно
108 void EXTI15_10_IRQHandler (void)
109 {
110     unsigned int i;
111     // Мигаем третьим диодом PG8 медленно 30 раз
112     for (i = 0; i < 30; ++i) {
113         GPIOG->ODR ^= GPIO_ODR_OD8;
114         delay (300000);
115     }
116
117     // Сбрасываем флаг прерывания для линии 15
118     // Пишем 1 в бит PR15, подтверждая выполнение
119     EXTI->PR |= EXTI_PR_PR15;
120 }

```

Listing 1: Основная функция программы

Заключение

В ходе работы изучены механизмы обработки прерываний в STM32 и принципы работы NVIC. Реализована программа, использующая кнопки "WAKEUP" и "USER" для управления светодиодами с разными приоритетами. Получены практические навыки конфигурирования периферийных модулей микроконтроллера и работы с прерываниями.